A

Mini Project Report

On

AWS LAMBDA FOR LOG PROCESSING WITH DYNAMODB

Submitted to JNTU HYDERABAD

In Partial Fulfilment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY

Submitted
By

MOHAMMED ARSH KHAN	**(228R1A1294)**
KURMA SRAVAN	**(228R1A1291)**
LANKA AKSHAYA	**(228R1A1292)**
THALLAPALLY SINDU	**(228R1A12C1)**

Under the Esteemed guidance of

Mrs. K. SUSHMA

Assistant Professor, Department of IT

Department of Information Technology

CMR ENGINEERING COLLEGE
(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU, Hyderabad)

(Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)

(2025-2026)

CMR ENGINEERING COLLEGE
(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU, Hyderabad)

(Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)

Department of Information Technology

CERTIFICATE

This is to certify that the project entitled **"Aws lambda for log processing with dynamodb"** is a bonafide work carried out by

MOHAMMED ARSH KHAN	**(228R1A1294)**
KURMA SRAVAN	**(228R1A1291)**
LANKA AKSHAYA	**(228R1A1292)**
THALLAPALLY SINDU	**(228R1A12C1)**

in partial fulfilment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **INFORMATION TECHNOLOGY** from CMR Engineering College, affiliated to JNTU, Hyderabad, under our guidance and supervision.

The results presented in this project have been verified and are found to be satisfactory. The results embodied in this project have not been submitted to any other university for the award of any other degree or diploma.

Internal Guide
Mrs. K. Sushma
Assistant Professor
Department of IT
CMREC, Hyderabad

Head of the Department
Dr. MADHAVI PINGILI
Professor & HOD
Department of IT
CMREC, Hyderabad

DECLARATION

This is to certify that the work reported in the present project entitled **"Aws lambda for log processing with dynamodb"** is a record of bonafide work done by us in the Department of Information Technology, CMR Engineering College, JNTU Hyderabad. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

MOHAMMED ARSH KHAN (228R1A1294)

KURMA SRAVAN (228R1A1291)

LANKA AKSHAYA (228R1A1292)

THALLAPALLY SINDU (228R1A12C1)

ACKNOWLEDGEMENT

We are extremely grateful to **Dr. A. Srinivasula Reddy**, Principal and **Dr. Madhavi Pingili**, HOD, **Department of IT, CMR Engineering College** for their constant support.

We are extremely thankful to **Mrs. K. SUSHMA,** Assistant Professor, Internal Guide, Department of IT, for his constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if we do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and co-ordination extended in bringing out this project successfully in time.

Finally, We are very much thankful to our parents who guided us for every step.

MOHAMMED ARSH KHAN (228R1A1294)

KURMA SRAVAN (228R1A1291)

LANKA AKSHAYA (228R1A1292)

THALLAPALLY SINDU (228R1A12C1)

CONTENTS

TOPIC	PAGE NO
ABSTRACT	**I**
LIST OF FIGURES	**II**
LIST OF TABLES	**III**
1. INTRODUCTION	**4**
1.1. Introduction & Objectives	4
1.2. Project Objectives	5
1.3. Purpose of the project	5
1.4. Existing System with Disadvantages	5
1.5. Proposed System With features	6
1.6. Input and Output Design	8
2. LITERATURE SURVEY	**10**
3. SOFTWARE REQUIREMENT ANALYSIS	**13**
3.1. Problem Specification	13
3.2. Modules and their Functionalities	13
3.3. Functional Requirements	13
3.4. Non-Functional Requirements	14
3.5. Feasibility Study	14
4. SOFTWARE & HARDWARE REQUIREMENTS	**17**
4.1. Software requirements	17
4.2. Hardware requirements	18
5. SOFTWARE DESIGN	**19**
5.1. System Architecture	19
5.2. Dataflow Diagrams	20
5.3. UML Diagrams	21

6. CODING AND IMPLEMENTATION **26**

6.1. Source code 26

6.2. Implementation 32

6.2.1. HTML 32

7. SYSTEM TESTING **34**

7.1. Types of System Testing 34

7.2. Test Cases 37

8. OUTPUT SCREENS **40**

9. CONCLUSION **44**

10. FUTURE ENHANCEMENTS **45**

11. REFERENCES **46**

ABSTRACT

Real-time processing of logs has become critical in the cloud-native world today, particularly in applications that produce large system and application logs. AWS Lambda, with Amazon DynamoDB, provides a robust serverless architecture to process and handle such logs at scale. This serverless solution minimizes the necessity for manual intervention or infrastructure management, thus enabling organizations to monitor and respond to system events near real-time. AWS Lambda is automatically invoked for log events from sources such as Amazon CloudWatch, S3, or Kinesis, processing and filtering the data of interest prior to storing it in DynamoDB for further analysis and monitoring.The application of Lambda and DynamoDB offers various benefits to organizations in need of scalability, cost-effectiveness, and operational responsiveness. First, the event-driven aspect of AWS Lambda means logs are processed immediately they are generated, without waiting for batch jobs as in legacy batch-processing environments. This speed of response is paramount in settings where real-time alerting and anomaly detection have the potential to reduce downtime or security breaches. In addition, with DynamoDB's extremely scalable NoSQL database, the system is able to support variable amounts of log data without having to manually provision resources or concern itself with capacity constraints.An additional principal advantage is automation and standardization of log handling, which eliminates human error and maintains consistent processing logic. Logs can be automatically filtered, enriched, or grouped according to pre-defined rules, facilitating efficient querying and analysis for auditing, compliance, or performance tuning. The method also accommodates pay-per-use billing models, making it an economical solution even for organizations running with tight budgets or utilizing the AWS Free Tier. In total, the combination of AWS Lambda and DynamoDB is a strong and scalable approach to current log processing requirements, allowing teams to keep visibility and control over intricate distributed systems.

Keywords:

AWS Lambda, real-time log processing, Amazon DynamoDB, serverless architecture, event-driven computing, CloudWatch, automated monitoring, scalable NoSQL, Free Tier, anomaly detection

LIST OF FIGURES

S.NO	FIGURE NO	DESCRIPTION	PAGENO
1	1.5.1	Block diagram of proposed system	7
2	5.1	System Architecture	19
3	5.2	Data Flow diagram	20
4	5.3.1	Sequence diagram	22
5	5.3.2	Use case diagram	23
6	5.3.3	Activity diagram	24
7	5.3.4	Class diagram	25
8	7.2.2	Test Case 1	38
9	7.2.3	Test Case 2	38
10	7.2.4	Test Case 3	38
11	7.2.5	Test Case 4	39
12	7.2.6	Test Case 5	39
13	7.2.7	Test Case 6	39
14	8.1	Output Screen-1	40
15	8.2	Output Screen-2	40
16	8.3	Output Screen-3	41
17	8.4	Output Screen-4	41
18	8.5	Output Screen-5	42
19	8.6	Output Screen-6	42
20	8.7	Output Screen-7	43
21	8.8	Output Screen-8	43

LIST OF TABLES

S.NO	TABLE NO	DESCRIPTION	PAGENO
1	7.2	Test Cases	37

1. INTRODUCTION

1.1 Introduction

In the modern digital environment, where applications produce enormous amounts of operating and diagnostic data, efficient log processing is essential to keep the system healthy, secure, and performant. Conventional log processing products tend to use provisioned servers, intricate configurations, and batch-processing methods, which can introduce additional latency, increased operational expense, and scalability issues. To bypass these constraints, current architectures are increasingly embracing serverless computing paradigms—most prominently via AWS Lambda, complemented by Amazon DynamoDB.

AWS Lambda, an entirely managed serverless compute service, allows for real-time, event-triggered log processing without provisioning or managing infrastructure. AWS Lambda automatically runs custom code in response to certain triggers, for example, when a new log entry shows up in Amazon CloudWatch, S3, or Kinesis Streams. These logs can then be parsed, filtered, and formatted within milliseconds and be stored in Amazon DynamoDB, a highly scalable and fast NoSQL database service optimized for low latency and high availability.

This serverless integration presents huge benefits: real-time responsiveness, auto-scaling, and pay-per-use pricing, well-matched with current application requirements and cost limitations—particularly for organizations making use of the AWS Free Tier. Also, by separating compute from storage and automating the whole log ingestion pipeline, this structure removes complexity and supports a modular, maintainable system design.

The application of AWS Lambda and DynamoDB for log processing not only is effective but also forms the basis of observability building, audit trails support, anomaly detection, and proactive system monitoring. This project investigates the use of such a system, its architecture, usage, advantages, and real-world applications in a cloud-native ecosystem.

1.2 Project Objectives

The objective of this project is to design and implement a **serverless log processing system** using **AWS Lambda** and **Amazon DynamoDB** to enable real-time ingestion, parsing, and storage of application logs. By leveraging the event-driven nature of AWS Lambda, the system aims to automate log handling tasks such as error detection, categorization, and anomaly tracking without the need for manual intervention or dedicated infrastructure. The integration with DynamoDB ensures scalable, low-latency storage and querying of processed log data, supporting efficient monitoring, alerting, and auditing for cloud-based applications.

1.3 Purpose of the Project

The purpose of this project is to provide an efficient, scalable, and cost-effective solution for real-time log processing in cloud environments by utilizing AWS Lambda and Amazon DynamoDB. Traditional log handling methods often involve manual processes, fixed infrastructure, and delays in identifying critical issues. This project aims to eliminate those limitations by introducing a **serverless architecture** that ensures automatic log ingestion, fast processing, and immediate storage. The system is intended to improve application observability, enhance operational responsiveness, and support better decision-making through reliable and timely log data.

1.4 Existing System with Disadvantages

Traditional log processing systems often rely on manual scripts or provisioned servers to ingest and analyze logs, which can be **time-consuming, difficult to scale**, and **inefficient in responding to real-time events**. In cloud-native applications generating large volumes of logs, developers and operations teams may find it challenging to **quickly identify and respond to critical issues**, leading to delays in troubleshooting and reduced system reliability.

Disadvantages

- The **absence of automation** in log ingestion and processing introduces latency, especially in detecting and acting on anomalies, errors, or unauthorized access patterns.
- Manual log monitoring increases the **operational burden on developers**, limiting their ability to focus on performance tuning, innovation, or security enhancements.

1.5 Proposed System with Features

The proposed **Serverless Log Processing System** aims to overcome the limitations of traditional log management by leveraging the power of **AWS Lambda** for event-driven execution and **Amazon DynamoDB** for scalable, real-time data storage. This system introduces **automation at every stage of the log handling pipeline**, enabling real-time ingestion, parsing, storage, and monitoring of logs from various cloud-based applications and services.Key features include **automated log ingestion** triggered by services like **CloudWatch, S3, or Kinesis**, ensuring logs are captured and processed as soon as they are generated. The system parses logs to extract relevant information such as error types, timestamps, user actions, and system responses, and then **stores the structured data in DynamoDB**, making it easily searchable and available for audit trails or analytics.Additionally, the system supports **custom filtering and categorization**, allowing logs to be sorted by severity, source, or type, which significantly improves troubleshooting and monitoring efficiency. Integration with **AWS SNS or other alerting mechanisms** enables real-time notifications in case of anomalies or critical failures, helping teams respond to incidents faster.The centralized, serverless architecture also ensures **high scalability and minimal operational overhead**, as there's no need to manage infrastructure or worry about provisioning. The solution operates on a **pay-as-you-go model**, making it ideal for organizations operating within limited budgets, including those on the **AWS Free Tier**.Furthermore, the system enhances observability and security by providing a **clear and queryable audit trail** of application events, improving visibility into system behavior over time. The modular design supports easy expansion, allowing additional processing logic, analytics dashboards, or third-party integrations to be added as needed.By automating log handling and enabling real-time insights, the proposed system **empowers development and DevOps teams** to proactively manage application performance, detect issues early, and maintain robust operational health across distributed cloud environments.

SCRIPTING LANGUAGE:

- **Serverless architecture eliminates the need to manage infrastructure**, allowing developers to focus solely on code logic while AWS handles provisioning, scaling, and maintenance.
- **AWS Lambda functions are event-driven**, automatically triggered by sources like CloudWatch Logs, S3, or Kinesis whenever new log data is available.
- **Amazon DynamoDB is a highly scalable NoSQL database**, used to store and query structured log data with low latency, making it ideal for real-time applications.

- **The system is platform-independent and cloud-native**, meaning it can be deployed in any environment that supports AWS services without modification.
- **Processing is asynchronous and scalable by design**, enabling high-throughput log ingestion and analysis without delays, even during traffic spikes.
- **Logs are parsed, filtered, and categorized in real-time** by Lambda functions using built-in logic or external libraries, allowing efficient error tracking, auditing, and alerting.
- **Security and permissions are managed via AWS IAM**, ensuring that only authorized services and users can access or modify log data.
- **Typical use cases include real-time error detection, audit log tracking, and anomaly detection**, especially in microservices or distributed cloud applications.
- **The system supports modular integration** with services like AWS SNS, Elasticsearch, or third-party monitoring tools, making it easy to extend capabilities over time.

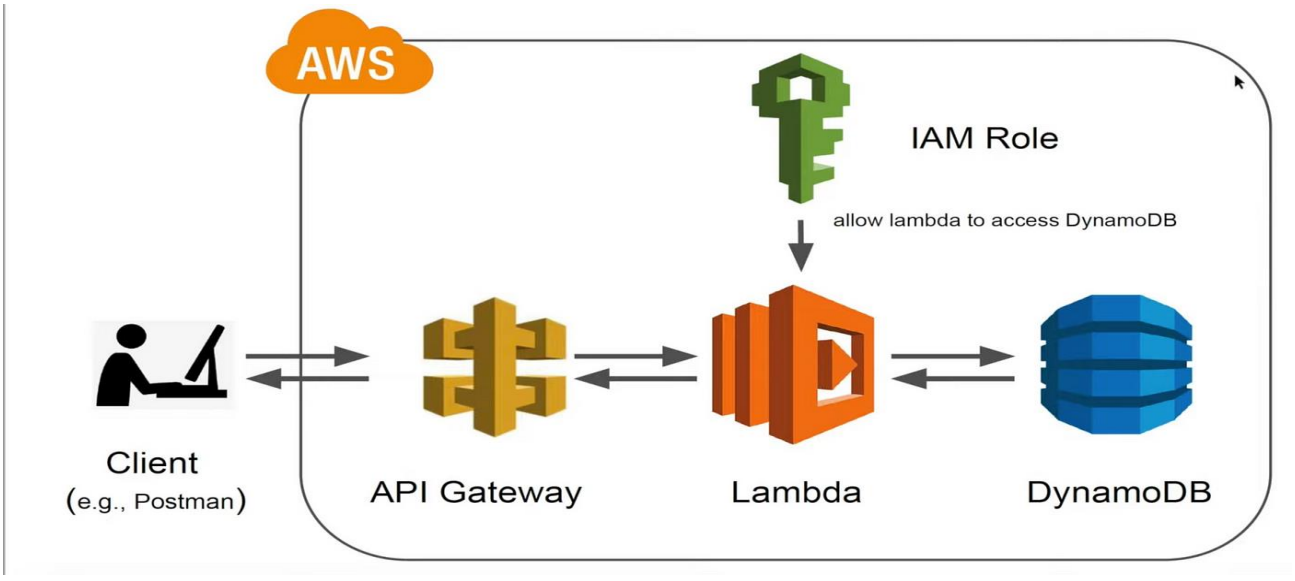


Figure 1.5.1: Block diagram of proposed system.

Advantages

• Serverless & Scalable

AWS Lambda automatically scales based on the number of incoming logs, so you don't have to manage servers or worry about capacity.

• Cost-Effective

You only pay for what you use. Lambda and DynamoDB are both part of AWS Free Tier (up to certain limits), perfect for small apps and learning.

• Real-Time Processing

Logs are processed instantly as they are received—no need to schedule tasks or run background jobs manually.

1.6 Input And Output

1.7 Design Input Design

The input design for the **Log Processing System** involves the collection and processing of structured log data generated from a web-based front end. This input begins with the transmission of JSON-formatted log entries via secure API requests to an AWS API Gateway endpoint. Each log entry includes standardized fields such as **log level**, **message**, **source**, and **timestamp**. This consistent structure ensures accurate and uniform data ingestion.To maintain input flexibility and compatibility, the system supports logging from multiple sources such as web browsers, client-side applications, or backend services. Each request is processed by an AWS Lambda function, which serves as the core processing unit. The Lambda function validates the incoming data, assigns a unique log identifier (UUID), appends a server-side timestamp if necessary, and then stores the structured log entry into a **DynamoDB table**.The input design also supports CORS (Cross-Origin Resource Sharing) configurations through API Gateway, allowing secure log submissions directly from client-side scripts. Optional fields such as **user-agent**, **IP address**, or **application module** can be added to enhance the richness of the logged data.This approach ensures minimal manual intervention, promotes consistency across all log entries, and facilitates efficient storage and retrieval of logs for further analysis or monitoring. By automating the data intake pipeline, the system achieves high availability, scalability, and reliability for real-time log processing needs.

Objectives

- **Client-side applications** should be able to generate detailed log entries containing essential fields such as **log level**, **source**, **message content**, and **timestamp**. These fields are designed to ensure each log entry is standardized, searchable, and informative for further analysis.
- The web application must **transmit logs securely** to an AWS **API Gateway** endpoint using HTTP POST requests. The system should be capable of processing logs from different environments such as browsers or servers. Logs must be accepted in **JSON format** and parsed accurately by the backend **Lambda function**, which stores the data in a **DynamoDB** table.
- A flexible configuration for **Cross-Origin Resource Sharing (CORS)** should be implemented, allowing the frontend to communicate with the backend seamlessly, regardless of origin. This ensures a responsive user experience and secure transmission of logs from any supported client.

Output Design

The output design of the **AWS Log Management System** is carefully structured to ensure clear visibility into user interactions, application behavior, and error reporting. This system centralizes log data and presents it in a format that enables **developers, testers, and administrators** to gain rapid and actionable insights.

The application produces **structured log entries** stored in **DynamoDB**, each enriched with metadata including timestamp, log level, message content, and source. These logs can be easily **searched and filtered**, supporting efficient debugging and system monitoring.

On the client side, the system delivers a **web-based log viewer**, automatically refreshed at intervals, that presents logs in categorized formats (e.g., INFO, WARNING, ERROR). This output allows users to quickly assess system health and monitor behavior in near real-time, directly from the browser.

The system supports **JSON-based API responses**, ensuring compatibility with external monitoring dashboards or alerting systems. This facilitates integrations with tools like **CloudWatch, Kibana, or custom analytics platforms** for deeper log analysis and trend visualization.

In the event of failed network requests, the application stores **temporary logs in local storage**, providing fallback mechanisms and ensuring no data is lost. These logs can later be retried and sent automatically once connectivity is restored, ensuring **data integrity** and **resilience**.

For transparency and traceability, the backend records all incoming log events along with response statuses and source details. These records help establish **audit trails** and ensure accountability, especially when diagnosing production issues or tracking user behavior.

Additionally, periodic reports can be generated summarizing **log volume trends, error spikes, and user activity** over time. These insights support **data-driven optimizations**, allowing teams to proactively enhance application stability and user experience.

Overall, this robust output design ensures that all stakeholders—from developers to managers—have access to meaningful, well-organized information, enabling **faster debugging**, **more informed decision-making**, and **improved operational oversight**.

2. LITERATURE SURVEY

The paradigm of serverless computing, particularly through platforms like AWS Lambda, has revolutionized the development of scalable, event-driven architectures. When combined with managed NoSQL databases such as Amazon DynamoDB and exposed through RESTful APIs using Amazon API Gateway, these architectures enable robust, low-latency, and highly available systems for processing, storing, and retrieving application logs. Numerous studies and publications in recent years underscore the benefits and methodologies surrounding this approach.

[1] Guo RongBin and Zhao XiuCai (2024)

In their work on automation in testing systems, the authors emphasized the importance of reusable software patterns to reduce development time and cost. This aligns closely with how AWS Lambda allows developers to deploy functions rapidly using pre-defined templates and event-driven triggers. In log processing, these patterns translate to automatically handling logs emitted from various AWS services (e.g., CloudWatch, S3, or application APIs), reducing manual monitoring efforts and providing immediate insights into system health.

[2] Yang Aibing, Sun Ye, and Peng Wei (2023)

These researchers explored how the concept of software-defined systems is replacing traditional hardware setups. AWS Lambda embodies this shift by eliminating the need for dedicated log-processing servers. Instead, it enables dynamic provisioning based on incoming traffic or event volume, which is essential for log-heavy environments. For example, a Lambda function can automatically trigger on new CloudWatch logs or incoming API calls, parse relevant log data, and store structured information in DynamoDB — all without provisioning any server manually.

[3] Q. Liu (2024)

Liu's research introduced a signal-based design model, which parallels how Lambda functions are invoked by event signals. In the context of your project, Lambda is triggered by events like API Gateway requests or log object uploads in S3. These events act as signals that initiate downstream workflows — such as transforming logs, validating entries, or enriching data — and storing them in DynamoDB for future retrieval via REST APIs.

[4] Y. P. Wang, D. G. Wang, and W. U. Zhong-De (2021)

Their study focused on modular test systems, promoting component interchangeability. Lambda-based architecture excels in modular design. Each Lambda function can handle a specific micro-task such as data validation, transformation, or enrichment. Coupled with REST APIs, it enables a clean separation of concerns where one endpoint could trigger log ingestion, another fetch filtered results, and another handle admin queries — all backed by a DynamoDB table.

[5] Liu Qi and He Yuzhu (2023)

Their work emphasized reliability and flexibility in real-time systems. AWS Lambda, with built-in retry mechanisms, version control, and cloud monitoring integrations (like CloudWatch Logs and X-Ray), provides robust tooling to build fault-tolerant logging systems. This ensures that even if one invocation fails (e.g., during log ingestion), the system can recover gracefully and maintain data integrity.

[6] Chen Heng (2022)

This study on resource monitoring systems directly correlates with the observability features of serverless log processing pipelines. Your system can utilize CloudWatch dashboards, AWS X-Ray for tracing, and DynamoDB Streams to track every state change or data update. These tools collectively provide a full-stack monitoring suite without additional configuration, ensuring visibility across the log pipeline from ingestion to output.

[7] K. Ellis (2021)

By proposing a standardized data handling framework, Ellis's work supports RESTful design principles. Your project mirrors this in the way logs are structured in DynamoDB — with primary keys, sort keys, and GSI indexes — to allow efficient querying and filtering through API Gateway endpoints. RESTful APIs are essential for enabling external services or frontend dashboards to access and manipulate log data in a controlled, secure, and standardized way.

[8] M. Zareappor, P. Shamsolmoali (2016)

Although primarily focused on fraud detection, this research discussed the use of ensemble techniques for data classification. The principles are applicable to your log processing project, where future enhancements could include analyzing logs using Amazon SageMaker or integrating machine learning Lambda layers to classify or tag logs based on severity, service, or potential anomalies.

[9] A. Y. Ng and M. I. Jordan (2021)

Their comparison of classifiers (logistic regression vs. naive Bayes) underscores the utility of intelligent decision-making within automation. Logs captured and processed by your Lambda functions could be augmented with prediction logic (e.g., tagging unusual error patterns or high-latency API responses), laying the groundwork for proactive monitoring and auto-remediation.

[10] S. Stolfo et al. (2019)

Their research into meta-learning and behavioral models further enhances the vision for your log-processing system. By extending your architecture to detect abnormal usage patterns from logs, you could enable security features like anomaly detection or automated alerts through AWS SNS or SES.

Recent research in automated and event-driven software systems supports the implementation of serverless architectures like AWS Lambda for log processing. Guo RongBin and Zhao XiuCai (2024) highlighted reusable software frameworks that reduce development costs — similar to AWS Lambda's modular design. Yang Aibing et al. (2023) focused on software-defined systems, aligning with Lambda's ability to respond to log-based triggers automatically.

Q. Liu (2024) and Liu Qi & He Yuzhu (2023) proposed signal-oriented processing, which resembles AWS Lambda's event-driven invocations. These models promote low-latency processing of logs triggered via REST APIs or S3 uploads. Wang et al. (2021) emphasized modular system components, resonating with the microservices architecture enabled by Lambda functions and RESTful endpoints.

Studies by Chen Heng (2022) and Ellis (2021) explored effective resource management and REST standardization, respectively, supporting structured storage and retrieval in DynamoDB. Ng & Jordan (2021) and Stolfo et al. (2019) introduced ML techniques applicable to log analytics for anomaly detection and decision-making. Altogether, the literature confirms that a Lambda + DynamoDB + API Gateway setup is an optimal, modern approach to scalable log processing and real-time data handling.

3. SOFTWARE REQUREIMENTS ANALYSIS

3.1 Problem Statement

The primary problem addressed by this project is the inefficiency in log data processing, storage, and retrieval in traditional systems. Many organizations rely on manual or outdated methods for log processing, leading to delays, errors, and difficulty in accessing real-time insights. As the volume of log data increases, these traditional approaches struggle to scale effectively, resulting in lost opportunities to identify critical issues, monitor performance, and ensure system security.With AWS Lambda, DynamoDB, and API Gateway, the project aims to automate log data ingestion, storage, and retrieval, reducing the reliance on manual interventions and improving system performance. The Lambda functions process logs in real time, triggering efficient data processing without the need for server management. DynamoDB serves as a highly scalable and low-latency data store, ensuring logs are stored in a structured and accessible way, while the REST API Gateway facilitates easy, secure, and consistent access to this data for further analysis or integration with other systems.

3.2 Modules and Their Functionalities

Data Preprocessing

The primary modules used in this AWS Lambda-based log processing project are **AWS Lambda** for event-driven log processing and automation, **Amazon DynamoDB** for scalable and low-latency log storage, **Amazon API Gateway** for exposing RESTful APIs to access log data securely, **Amazon CloudWatch** for monitoring and logging Lambda function performance, **Amazon S3** (optional) for storing raw log files, **AWS IAM** for managing secure access permissions to AWS resources, **Amazon SNS** (optional) for sending alerts and notifications, and **AWS Step Functions** (optional) for orchestrating complex workflows involving multiple AWS services. These modules work together to process, store, and retrieve log data efficiently, while ensuring scalability, security, and real-time access.
AWS Lambda: Executes code in response to events like log uploads, enabling real-time log processing and automation without server management.
Amazon DynamoDB: Provides a fast and scalable NoSQL database to store and query processed log data with low latency.
Amazon API Gateway: Exposes REST APIs to securely interact with the log data stored in DynamoDB, enabling easy access by external systems or users.

3.3 Non-Functional Requirements

The non-functional requirements for the AWS Lambda-based log processing system with DynamoDB and API Gateway focus on ensuring scalability, reliability, performance, security, usability, and compatibility. The system must scale efficiently to handle varying volumes of log data, automatically adjusting to peak loads without performance degradation, with AWS Lambda providing serverless scalability and DynamoDB supporting high throughput for log storage and queries. Reliability is paramount, with minimal downtime and robust failover mechanisms, leveraging AWS's multi-AZ replication for fault tolerance and automated backups to safeguard data. Performance must be optimized for fast response times across all user interactions, with Lambda ensuring real-time log processing and DynamoDB offering low-latency data retrieval even under heavy load. The system must also be secure, with strong encryption of log data both in transit and at rest, fine-grained access controls via AWS IAM, and compliance with relevant data protection regulations. Usability is a key factor, with clear and intuitive API Gateway endpoints for easy integration and usage, supported by detailed documentation to simplify interactions for developers. Lastly, the system should be compatible across multiple platforms, ensuring seamless access to log data from various devices and browsers through RESTful APIs, with Lambda functions maintaining flexibility in interacting with other AWS services across different environments. These requirements ensure that the system delivers a scalable, reliable, secure, and user-friendly solution for efficient log processing and management.

3.4 Feasibility Study

The feasibility study for implementing your project, an AWS Lambda-based log processing system with DynamoDB and API Gateway, evaluates the practicality and benefits across technical, operational, financial, economic, and social aspects. From a **technical feasibility** perspective, the project can be implemented using established AWS services such as Lambda for serverless computing, DynamoDB for scalable data storage, and API Gateway for secure API management. The required infrastructure and expertise are readily available, making the technical implementation achievable without significant barriers. Operationally, the system addresses critical challenges in log management, such as real-time processing, efficient data retrieval, and seamless integration into existing workflows. The solution will automate log ingestion, processing, and querying, significantly reducing manual workloads and providing valuable insights to system administrators and developers. Financially, the project is feasible due to the cost-effective nature of AWS services, which offer a pay-as-you-go model, reducing upfront investment

Economical Feasibility

The economic feasibility of the log processing system is supported by the low operational costs associated with using AWS serverless services. With a pay-as-you-go pricing model for AWS Lambda and DynamoDB, the system's expenses scale based on actual usage, making it an economical choice. The initial investment is limited to setting up the services, configuring Lambda functions, and building custom integrations, which is within budget as most of the required technologies are either free or low-cost. The cost of the project is justified by the benefits it provides, including faster log processing, real-time insights, reduced manual intervention, and overall improved efficiency. The system is expected to generate cost savings over time by reducing the resources needed for manual log management and improving troubleshooting and monitoring capabilities, ultimately leading to a higher return on investment for the organization.

Technical Feasibility

The technical feasibility of the log processing system using AWS Lambda, DynamoDB, and API Gateway is well-supported by existing technologies and infrastructure. The system can be implemented using AWS's serverless offerings, which reduce the need for managing physical servers, allowing for cost-effective scalability and reliability. AWS Lambda will process incoming log data in real time, while DynamoDB provides a highly scalable, low-latency data store for the logs, ensuring efficient storage and retrieval. API Gateway will expose secure APIs, enabling seamless interaction with the log data for both internal and external users. The technical expertise required to implement these services is readily available, and AWS's robust documentation and support ensure that the implementation process can proceed without significant barriers. Furthermore, the solution will not place excessive demands on available technical resources, ensuring that the system is sustainable and manageable by the client's technical team.

Social Feasibility

The social feasibility of the log processing system is primarily centered around user acceptance and ease of use. The system is designed to be intuitive and user-friendly, with simple API access to retrieve log data. The user interface will be easy to understand, and comprehensive documentation will be provided to facilitate onboarding and ensure that users, even with limited technical expertise, can efficiently interact with the system. The training process will focus on helping users understand how to query logs, monitor system performance, and use the API effectively. By making the system accessible and reducing the complexity of log management, the system is likely to be well-accepted by users. Additionally, the system is designed to be flexible, allowing users to provide feedback and suggestions for improvements, ensuring that their needs and concerns are addressed. This approach fosters a positive user experience and supports the successful adoption of the system.

4. SOFTWARE AND HARDWARE REQUIREMENTS

4.1 Software Requirements

The log processing system automates the ingestion, filtering, and querying of log data. The primary features include real-time log processing using AWS Lambda, storage of logs in DynamoDB for fast and scalable retrieval, and access to logs via RESTful APIs provided by API Gateway. The system supports querying logs by various parameters such as timestamp, severity, and content. Additionally, it includes functionality for setting up alerts based on specific log events and generating reports based on defined metrics.

Operating System and Environment:

- **Operating System**: Windows 10
- **Environment**: The system's backend services are hosted on the AWS cloud infrastructure, ensuring scalability, reliability, and high availability. The development environment on the client-side will be Windows 10, with tools such as Python for the Lambda functions and HTML for the frontend interface.

Coding Language:

- **Python**: The system will be developed using Python for the AWS Lambda functions. Python is chosen due to its ease of use, flexibility, and compatibility with AWS services like boto3 (AWS SDK for Python), which facilitates the integration of Lambda with DynamoDB and API Gateway.

Frontend:

- **HTML**: The frontend will be developed using HTML for creating a basic web interface that allows users to interact with the backend APIs. It will display logs, provide query functionality, and allow users to view processed log data in a user-friendly format.

Design Constraints:

- The system must be serverless, relying on AWS Lambda for log processing and handling log storage via DynamoDB.
- It must be scalable, able to handle varying volumes of logs without performance degradation, especially during peak usage periods.
- Security must be prioritized, ensuring that log data is encrypted and access is strictly controlled through IAM roles.
- The frontend interface should be simple, intuitive, and responsive to cater to users across different devices.

4.2 Hardware Requirements

The minimum hardware requirements for the log processing system, which utilizes AWS Lambda, DynamoDB, and API Gateway, are as follows:

System	:	Pentium Dual Core.
Hard Disk	:	120 GB.
Monitor	:	15'' LED
Input Devices	:	Keyboard, Mouse
Ram	:	4 GB

These hardware requirements are designed to support the development and testing environment for the log processing system. Since the system leverages cloud infrastructure for processing and storage, the local machine does not need to have high computational resources.

5. SOFTWARE DESIGN

5.1 System Architecture

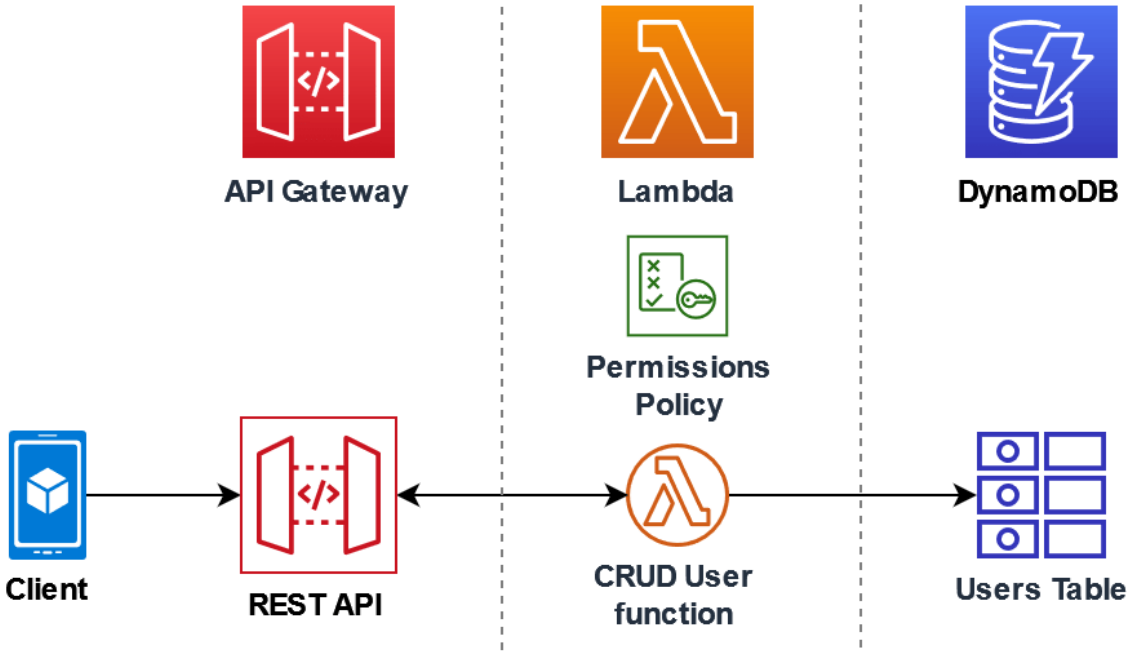


Figure:5.1 System Architecture

Traditional log management methods are often manual, time-consuming, and prone to errors, leading to delayed issue detection, inefficient troubleshooting, and the potential for overlooking critical events in system logs. Organizations that handle large volumes of log data struggle to effectively process and analyze each log entry, resulting in slower response times and a risk of missing key insights. Moreover, manual processes lack the ability to quickly filter and query logs, which can lead to inconsistent decision-making when identifying and resolving issues. This lack of real-time log analysis and automated alerting also complicates compliance with security and operational monitoring regulations. Additionally, the absence of centralized data storage and collaboration tools for log management further hampers team coordination, resulting in fragmented communication, duplicated efforts, and inefficient workflows, which ultimately impact the speed and effectiveness of incident response and troubleshooting.

5.2 Dataflow Diagram

The **Data Flow Diagram (DFD)**, also known as a **bubble chart**, is a graphical tool used to visualize the flow of information within a system. In the context of this project, the DFD illustrates how log data is collected, processed, stored, and accessed using serverless components such as AWS Lambda, DynamoDB, and REST APIs.

The DFD helps represent the **key components** of the system:

- **External entities** like users or applications sending log data.
- **Processes** such as data validation, parsing, and transformation carried out by AWS Lambda functions.
- **Data flows** that indicate the path of data movement from source to processing and then to storage or response.

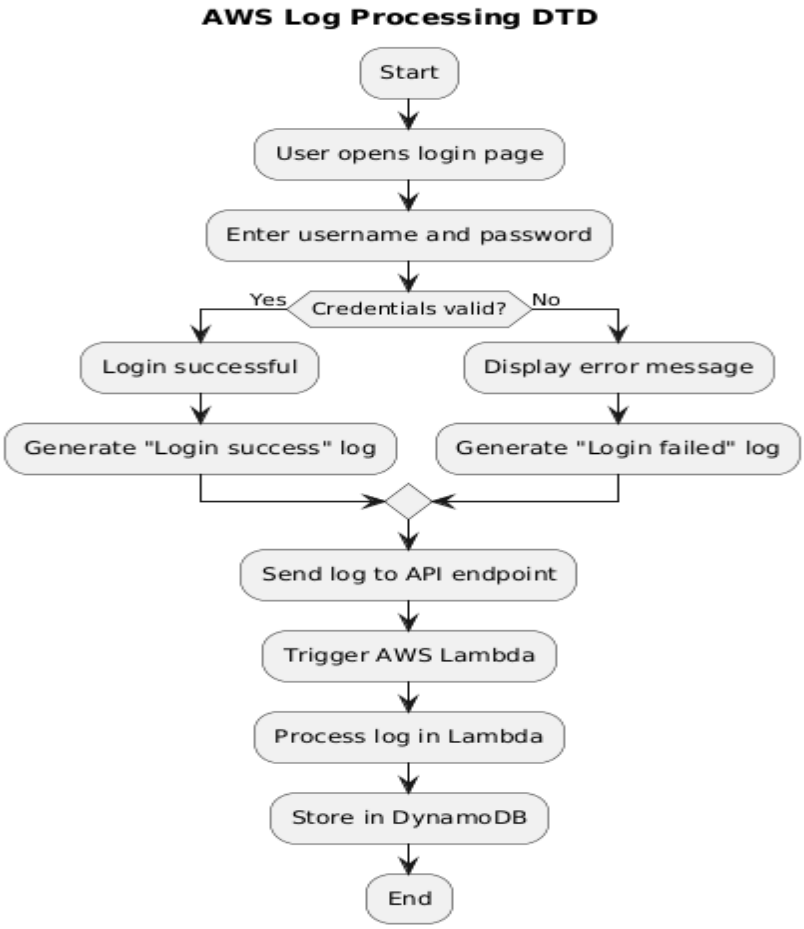


Figure:5.2 Dataflow Diagram

5.3 UML Diagrams

UML is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems. UML was created by the Object Management Group (OMG) and UML 1.0 specification draft was proposed to the OMG in January 1997.

There are several types of UML diagrams and each one of them serves a different purpose regardless of whether it is being designed before the implementation or after (as part of documentation). UML has a direct relation with object-oriented analysis and design. After some standardization, UML has become an OMG standard. The two broadest categories that encompass all other types are:

- Behavioral UML diagram and
- Structural UML diagram.

As the name suggests, some UML diagrams try to analyses and depict the structure of a system or process, whereas other describe the behavior of the system, its actors, and its building components.

Goals: The Primary goals in the design of the UML are as follows:

Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.

Provide a formal basis for understanding the modeling language.

Encourage the growth of OO tools market.

Integrate best practices.

The different types are as follows:

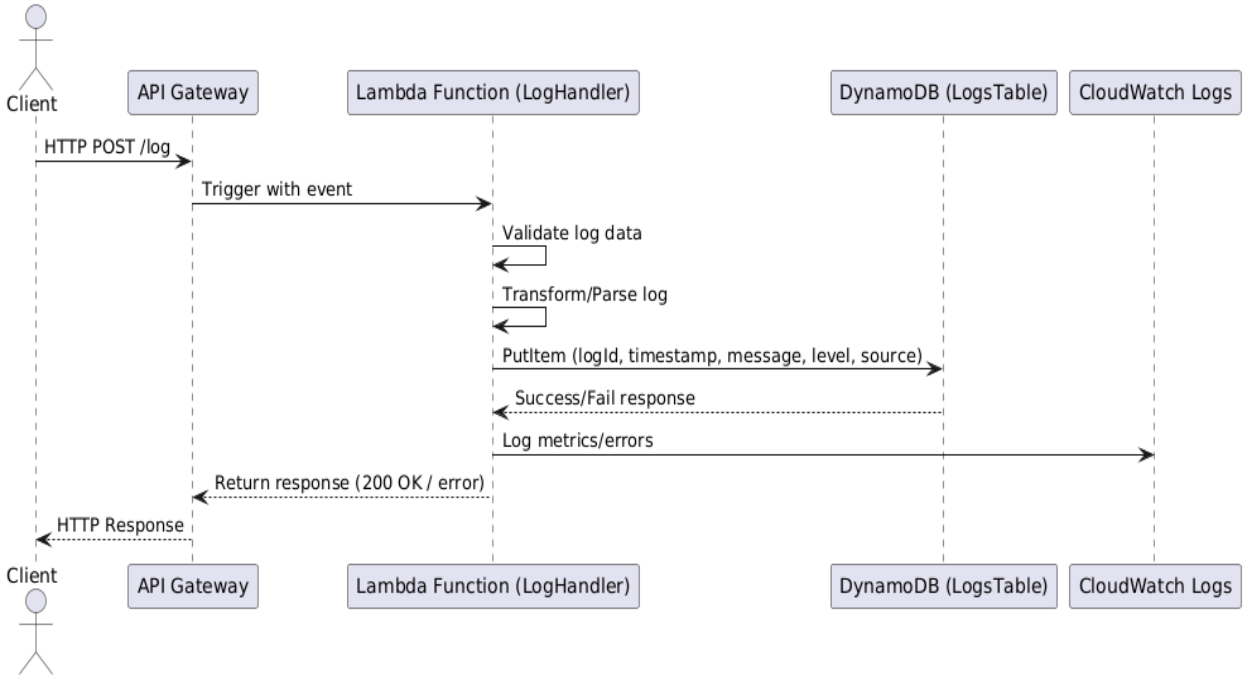
- Sequence diagram
- Use case Diagram
- Activity diagram
- Class diagram
- Collaboration diagram

Sequence Diagram

A sequence diagram simply depicts interaction between objects in a sequential order i.e., the order in which these interactions take place. We can also use the terms event diagrams or event scenarios to refer to a sequence diagram. Sequence diagrams describe how and in what order the objects in a system function. These diagrams are widely used by businessmen and software developers to document and understand requirements for new and existing systems.

Figure 5.3.1 Sequence Diagram

List of actions



User:

User need to press any of the given three (i.e., Prediction data, prediction Skills) then he will get the output accordingly.

System: System will give the output as he enters according to the given data.

Result: As per user enters the data it will give whether it is Fraud or not fraud

Use Case Diagram

A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use case in which the user is involved. A use case diagram is used to structure of the behavior thing in a model. The use cases are represented by either circles or ellipses.

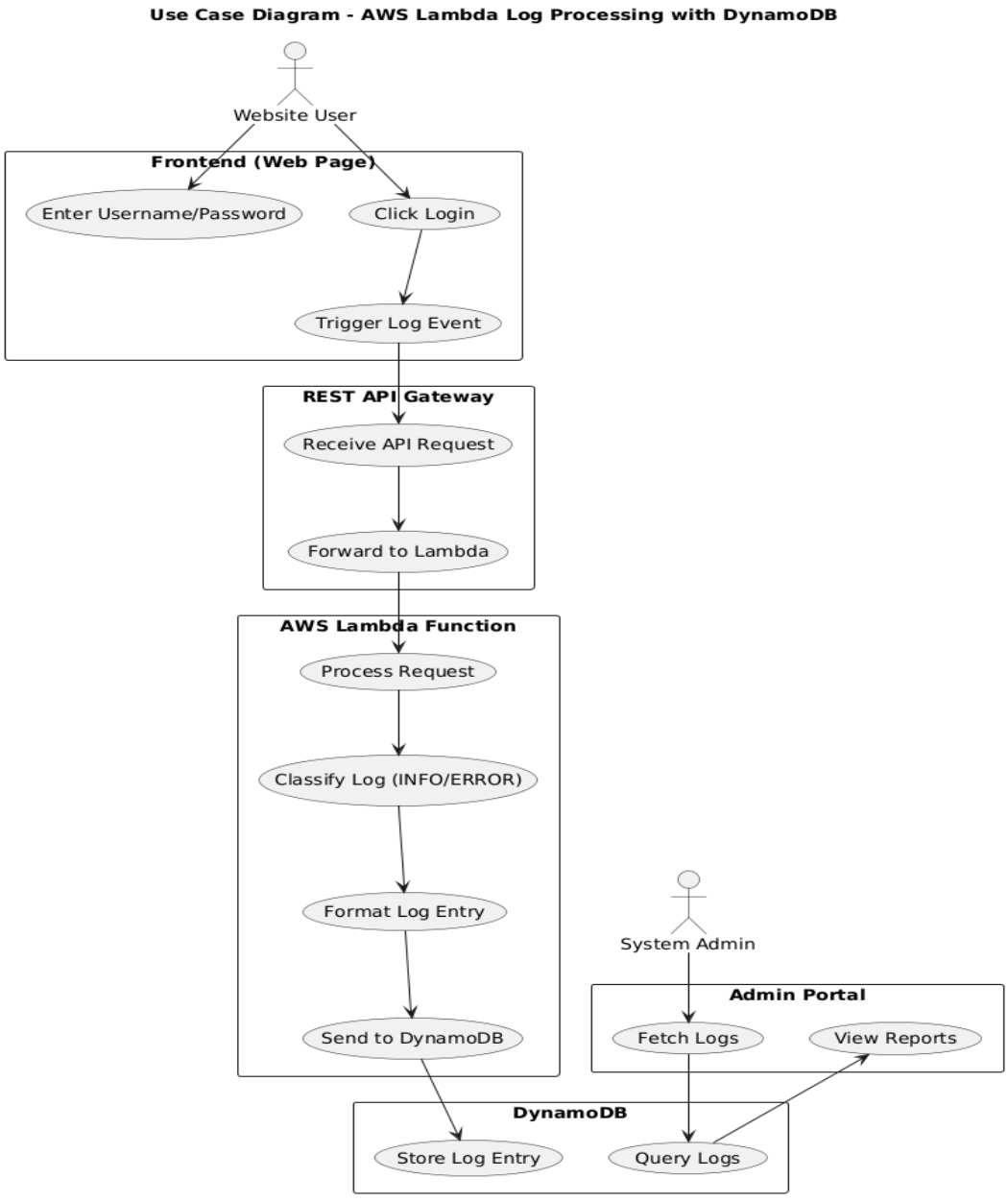


Figure 5.3.2 Use Case Diagram

Activity Diagram

Activity diagram is another important diagram in UML to describe the dynamic aspects of the system. Activity diagram is basically a flowchart to represent the flow from one activity to another activity. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc.

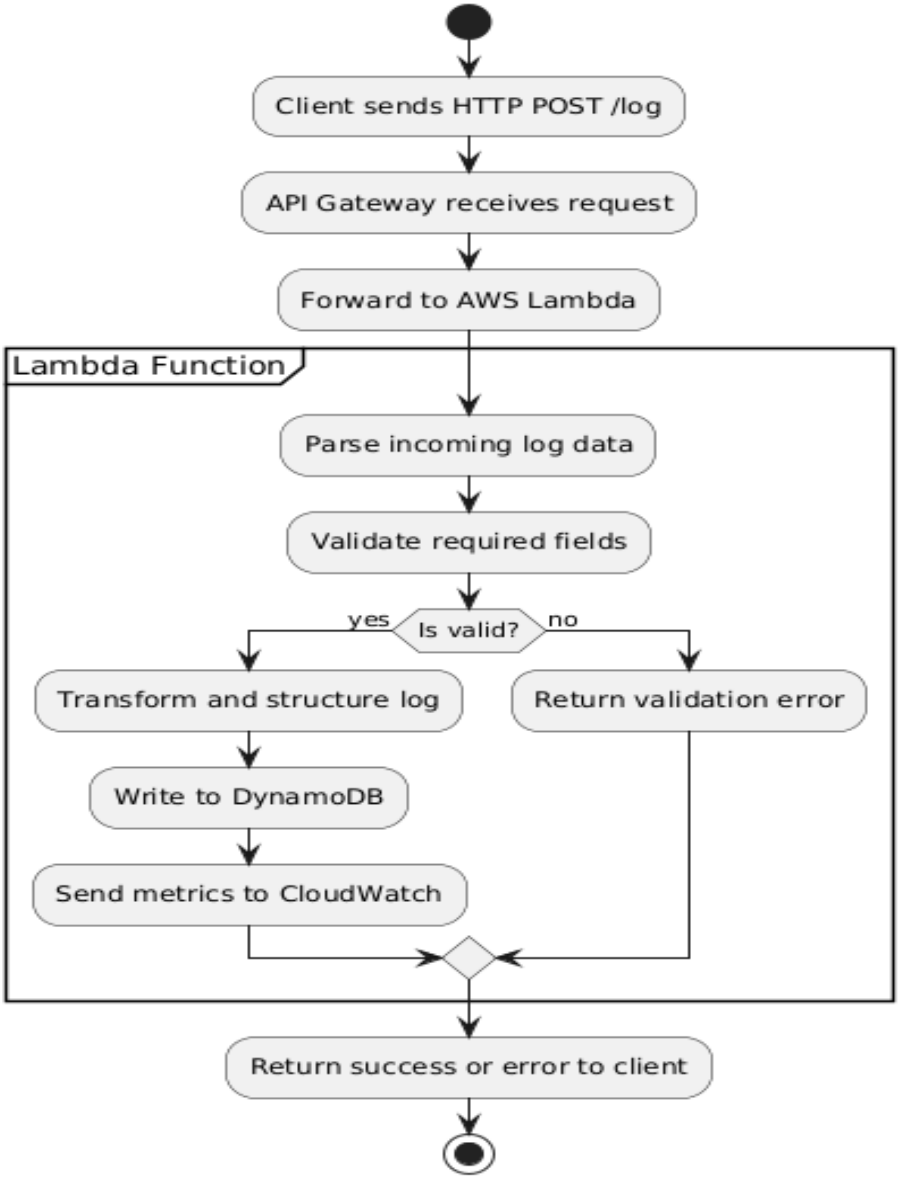


Figure 5.3.3 Activity Diagram

Class Diagram

In software engineering, a class diagram in the Unified Modelling Language (UML) is a type of staticstructure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

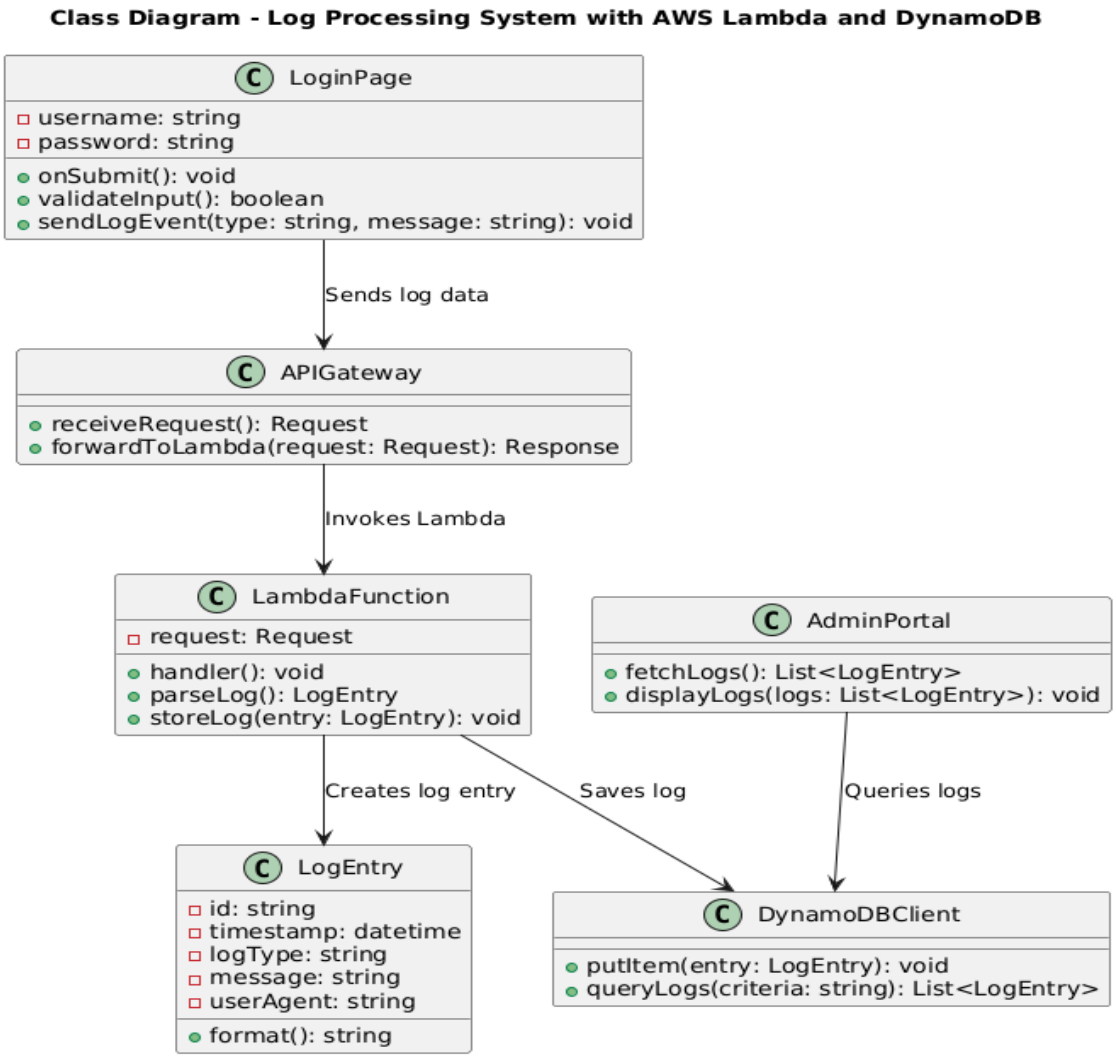


Figure 5.3.4 Class Diagram

6. CODING AND ITS IMPLEMENTATION

6.1 Source code of AWS Lambda

```
import json
import boto3
import uuid
from datetime import datetime

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('WebAppLogs')

def lambda_handler(event, context):
    print("EVENT RECEIVED:", json.dumps(event))  # Debug log

    try:
        # event is already the parsed body
        body = event  # No need to parse again

        item = {
            'LogId': str(uuid.uuid4()),
            'timestamp': datetime.utcnow().isoformat(),
            'level': body.get('level', 'INFO'),
            'source': body.get('source', 'web'),
            'message': body.get('message', 'No message')
        }

        table.put_item(Item=item)

        return {
            'statusCode': 200,
```

```
            'headers': {
                'Access-Control-Allow-Origin': '*',
                'Access-Control-Allow-Headers': '*',
                'Access-Control-Allow-Methods': 'POST, OPTIONS'
            },
            'body': json.dumps({'message': 'Log stored successfully'})
        }

    except Exception as e:
        print("ERROR:", str(e))  # Debug
        return {
            'statusCode': 500,
            'headers': {
                'Access-Control-Allow-Origin': '*',
                'Access-Control-Allow-Headers': '*',
                'Access-Control-Allow-Methods': 'POST, OPTIONS'
            },
            'body': json.dumps({'error': str(e)})
        }
```

Frontend Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Login with Logs</title>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&display=swap" rel="stylesheet">
  <style>
    body {
      font-family: 'Inter', sans-serif;
      background-color: #f4f4f4;
```

```
  margin: 0;
  padding: 0;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  flex-direction: column;
}
.box {
  background: white;
  padding: 2rem;
  border-radius: 10px;
  box-shadow: 0 4px 8px rgba(0,0,0,0.1);
  display: none;
  width: 300px;
}
.visible {
  display: block;
}
h2 {
  margin-top: 0;
}
input[type="text"],
input[type="password"] {
  width: 100%;
  padding: 0.5rem;
  margin: 0.5rem 0;
  border: 1px solid #ccc;
  border-radius: 5px;
}
button {
  width: 100%;
```

```
      padding: 0.5rem;
      background-color: #007bff;
      color: white;
      border: none;
      border-radius: 5px;
      cursor: pointer;
      margin-top: 1rem;
    }
    button:hover {
      background-color: #0056b3;
    }
    .error {
      color: red;
      margin-top: 0.5rem;
      font-size: 0.9rem;
    }
    #logOutput {
      max-height: 300px;
      overflow-y: auto;
      width: 100%;
      padding: 1rem;
    }
    .log-info { color: green; }
    .log-warning { color: orange; }
    .log-error { color: red; }
  </style>
</head>
<body>

  <div class="box visible" id="loginBox">
    <h2>Login</h2>
    <input type="text" id="username" placeholder="Username" />
```

```
    <input type="password" id="password" placeholder="Password" />
    <button onclick="handleLogin()">Login</button>
    <div class="error" id="errorMsg"></div>
  </div>

  <div class="box" id="dashboard">
    <h2>Dashboard</h2>
    <p id="greeting"></p>
    <button onclick="logoutUser()">Logout</button>
    <div id="logOutput">Loading logs...</div>
  </div>

  <script>
    const API_BASE = 'https://qh97oti3kg.execute-api.ap-south-1.amazonaws.com/prod';
const LOG_ENDPOINT = `${API_BASE}/sendlog`;

    function logEvent(level, message) {
      const logData = {
        level: level.toUpperCase(),
        message: message.slice(0, 500),
        source: 'browser'
      };

      fetch(LOG_ENDPOINT, {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(logData)
      }).catch(err => console.error("Logging failed:", err));
    }
```

```
function handleLogin() {
  const username = document.getElementById('username').value.trim();
  const password = document.getElementById('password').value.trim();
  const errorBox = document.getElementById('errorMsg');
  errorBox.textContent = '';

  if (!username || !password) {
    errorBox.textContent = 'Please enter both fields.';
    logEvent('INFO', 'Empty login attempt');
    return;
  }

  const stored = JSON.parse(localStorage.getItem('firstLogin'));

  if (!stored) {
    localStorage.setItem('firstLogin', JSON.stringify({ username, password }));
    localStorage.setItem('currentSession', 'true');
    logEvent('INFO', 'First-time login');
    showDashboard(username);
  } else if (username !== stored.username || password !== stored.password) {
    errorBox.textContent = 'Invalid credentials';
    logEvent('WARNING', `Failed login for ${username}`);
  } else {
    localStorage.setItem('currentSession', 'true');
    logEvent('INFO', 'Login success');
    showDashboard(username);
  }
}

function showDashboard(username) {
  document.getElementById('loginBox').classList.remove('visible');
  document.getElementById('dashboard').classList.add('visible');
```

```
    document.getElementById('greeting').textContent = `Welcome, ${username}`;
  }

  function logoutUser() {
    localStorage.removeItem('currentSession');
    document.getElementById('dashboard').classList.remove('visible');
    document.getElementById('loginBox').classList.add('visible');
  }

  window.onload = () => {
    const saved = JSON.parse(localStorage.getItem('firstLogin'));
    const session = localStorage.getItem('currentSession');
    if (saved && session === 'true') showDashboard(saved.username);
  };
 </script>
</body>
</html>
```

6.2 Implementation

6.2.1 HTML

HTML (Hyper Text Markup Language) is the standard language used to create and structure content on the World Wide Web. It provides the foundation for web pages by using a system of elements, also known as tags, to define different parts of a webpage, such as headings, paragraphs, images, links, tables, forms, and more. HTML is not a programming language but a markup language, which means it is used to annotate content to give it structure and meaning so that web browsers can render it properly. Each HTML document begins with a <!DOCTYPE html> declaration, which tells the browser the type of document to expect, followed by an <html> tag that contains the entire webpage content.

Inside the HTML document, the content is organized within the <head> and <body> sections. The

<head> section includes metadata about the document, such as the title, character set, and links

to external resources like CSS stylesheets and JavaScript files. The <body> section contains all the visible content of the webpage, including text, images, videos, and other multimedia elements. HTML tags are typically paired, with an opening tag (e.g.,) and a closing tag (e.g.,), enclosing the content they apply to. Some tags, like <img> for images and
 for line breaks, are self-closing and do not require a closing tag.

One of the key strengths of HTML is its ability to create links between documents through anchor tags(<a>), enabling users to navigate between web pages via hyperlinks. Attributes within HTML tags provide additional information or modify the default behavior of elements. For example, the href attribute in an <a> tag specifies the URL of the linked document, while the src attribute in an <img> tag specifies the path to an image file.HTML also supports the inclusion of multimedia and interactive elements, such as audio, video, and embedded content from other web platforms. However, HTML alone does not handle the presentation or behavior of web content; it is often used in conjunction with CSS (Cascading Style Sheets) and JavaScript. CSS is used to control the visual presentation of the page, such as layout, colors, and fonts ,while JavaScript adds interactivity, such as form validation, dynamic content updates, and animations.HTML has evolved significantly since its inception, with HTML5 being the latest major version. HTML5 introduced new elements and attributes that enhance the capabilities of web pages, such as semantic elements (<header>, <footer>, <article>, <section>) for better content organization and new APIs for handling multimedia, graphics, and offline storage. These advancements have made HTML more powerful, allowing developers to create richer and more accessible web experiences. Overall, HTML remains the cornerstone of web development, providing the essential structure and foundation for all websites and web applications.HTML (Hyper Text Markup Language) continues to be the foundational technology that powers the vast majority of websites and web applications, making it an essential tool for web developers. It enables the creation of structured documents by defining elements such as headings, paragraphs, links, lists, images, forms, and other types of content, which are essential for displaying information on the web. By using a combination of HTML tags and attributes, developers can control the layout and appearance of content, ensuring it is presented in a way that is both user-friendly and accessible across different devices and browsers.HTML is designed to be simple and straightforward, which makes it easy to learn, even for beginners. This simplicity, however, does not limit its capabilities. HTML5, the latest version of the language, introduced a variety of new features and elements that enhance functionality and provide a more robust platform for developing modern web applications. These include semantic tags like <nav>, <aside>,<figure>, and <figcaption>, which not only improve the accessibility of web pages by providing more meaningful markup but also help search engines better understand the content of a page, thereby enhancing SEO (Search Engine Optimization).

7. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. System testing plays a crucial role in verifying the end-to-end functionality of the AWS Lambda-based log processing system. This process ensures that the integration of web interface, API Gateway, Lambda functions, and DynamoDB works seamlessly under expected and edge-case scenarios.

7.1. Types Of Tests

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the app.
Unit testing focuses on individual components like the login script and Lambda handler functions. For example, a unit test for login validation might take input: username="admin", password="1234", and return output: "Login Success" or "Invalid credentials". These tests use mock inputs and simulate Lambda environment variables.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. This verifies the correct interaction between system modules. For instance, submitting a wrong password in the frontend should trigger an error log stored in DynamoDB. Inputs involve front-end form actions, and the output is verified in the DynamoDB logs table.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.
Invalid Input : identified classes of invalid input must be rejected.
Functions : identified functions must be exercised.
Output : identified classes of log outputs must be exercised.
Procedures : interfacing systems or procedures must be invoked.

Functional testing checks if the system behaves as expected against its specifications. A test could involve entering a valid login, and the expected output would be a redirection to the dashboard and a success log saved.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. This includes testing the system in a production-like environment. All APIs, log handling, and database operations are tested. Inputs include valid and invalid login attempts, and outputs include log generation, Lambda execution, and DynamoDB storage.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level. Here, internal code logic is tested. It involves test cases like ensuring specific branches in the Lambda code execute for specific log levels (INFO, ERROR, etc.). Inputs are crafted to trigger all logical paths.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. This focuses on input-output without internal knowledge. A user submits a login form, and the system should store logs appropriately or redirect based on credentials.

Test strategy and approach:

-Top-down integration for testing frontend to backend
- Mocks and stubs for testing Lambda and API Gateway without incurring AWS charges
- Use of tools like Postman for API testing
- Use of AWS CloudWatch for monitoring logs in real-time
- Repeated execution under various network conditions

Test objectives:

- Verify Log Event Triggering: Ensure that submitting the login form on the frontend correctly triggers a log event and sends it to the backend API.
- Validate API Gateway Handling: Confirm that the REST API Gateway accurately receives and forwards the request to the AWS Lambda function.
- Check Lambda Log Classification: Test that the AWS Lambda function correctly classifies log entries as INFO or ERROR based on the request content.
- Confirm Data Storage in DynamoDB: Ensure that the processed and formatted log entries are successfully stored in the correct DynamoDB table with appropriate structure.
- Ensure Admin Access to Logs: Verify that the Admin Portal can correctly query, fetch, and display logs from DynamoDB using relevant filters or report views.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements. This determines whether the system meets client/user requirements. Example: If a client expects logs to be captured for all failed logins, the input (wrong credentials) should result in visible logs in the DB upon verification.

7.2 Test Cases:

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
TC01	Successful login	Valid username and password	Login success, INFO log stored	Login success, INFO log stored	Passed
TC02	Failed login (wrong username)	Invalid username, valid password	Login failure, WARNING log stored	Login failure, WARNING log stored	Passed
TC03	Failed login (wrong password)	Valid username, wrong password	Login failure, WARNING log stored	Login failure, WARNING log stored	Passed
TC04	Empty input fields	No username/password	Error message, INFO log stored	Error message, INFO log stored	Passed
TC05	First-time login	New username and password	Login success, INFO log stored	Login success, INFO log stored	Passed
TC06	Log event on logout	Click logout	Session ends, INFO log stored	Session ends, INFO log stored	Passed
TC07	Multiple wrong attempts	3 wrong tries	Multiple WARNING logs stored	Multiple WARNING logs stored	Passed
TC08	Database storage test	Trigger log via action	Log stored in DynamoDB	Log stored in DynamoDB	Passed
TC09	Log formatting test	Trigger malformed log	Handled gracefully	Handled gracefully	Passed
TC10	Network issue simulation	Block API call	Fails silently or retries	Fails silently or retries	Failed

Table no 7.2 Test Cases

Test Case 1:

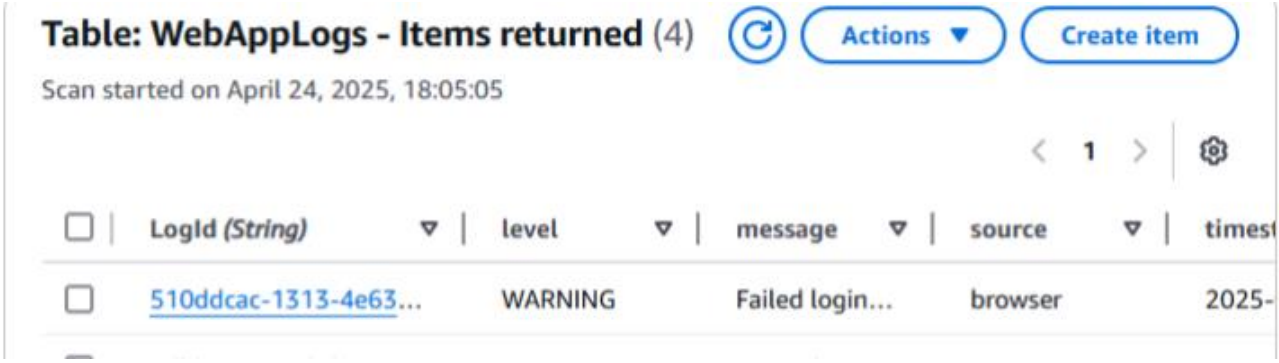


Figure 7.2.2: Test Case 1

Test Case 2:

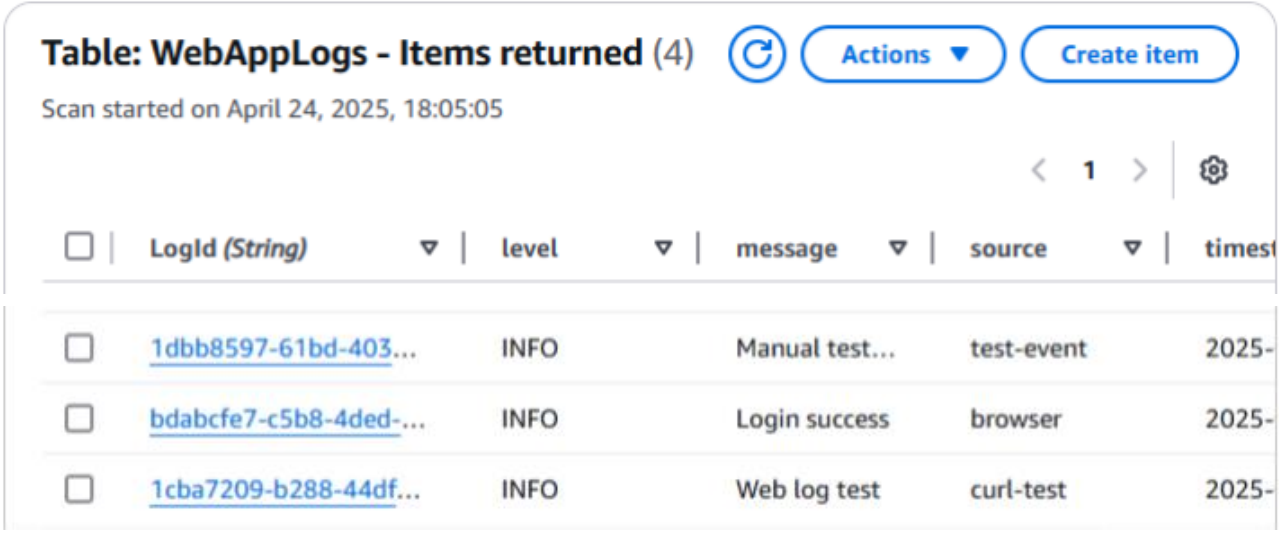


Figure 7.2.3: Test Case 2

Test Case 3:

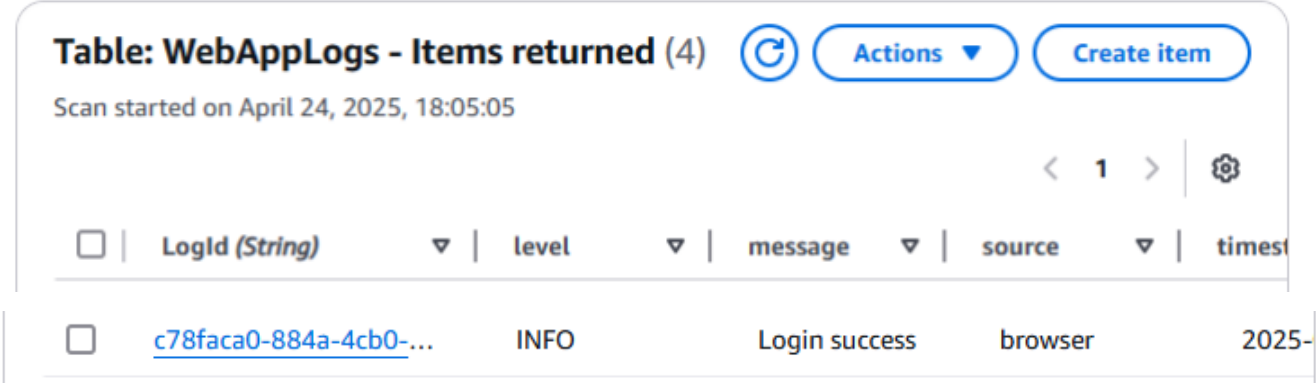


Figure 7.2.4: Test Case 3

Test Case 4:

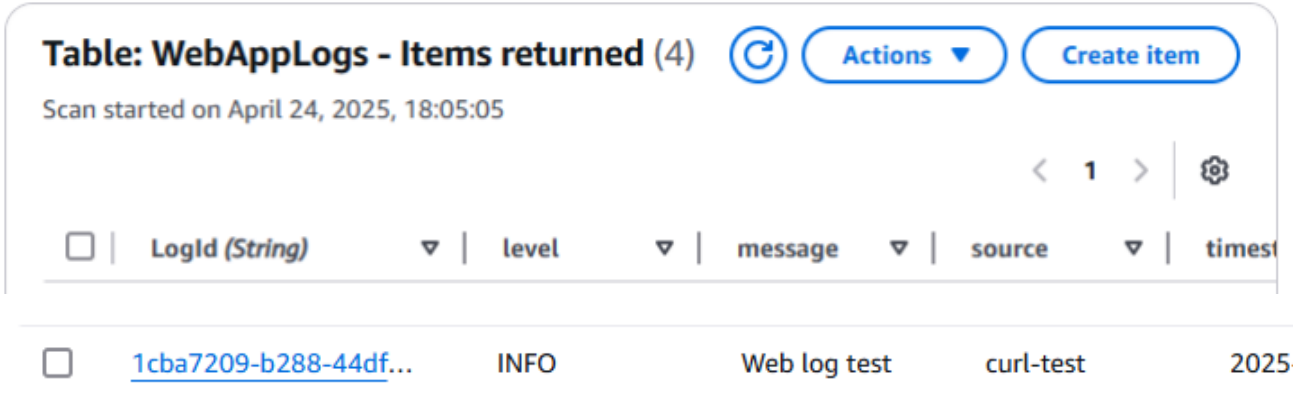


Figure 7.2.5: Test Case 4

Test Case 5:

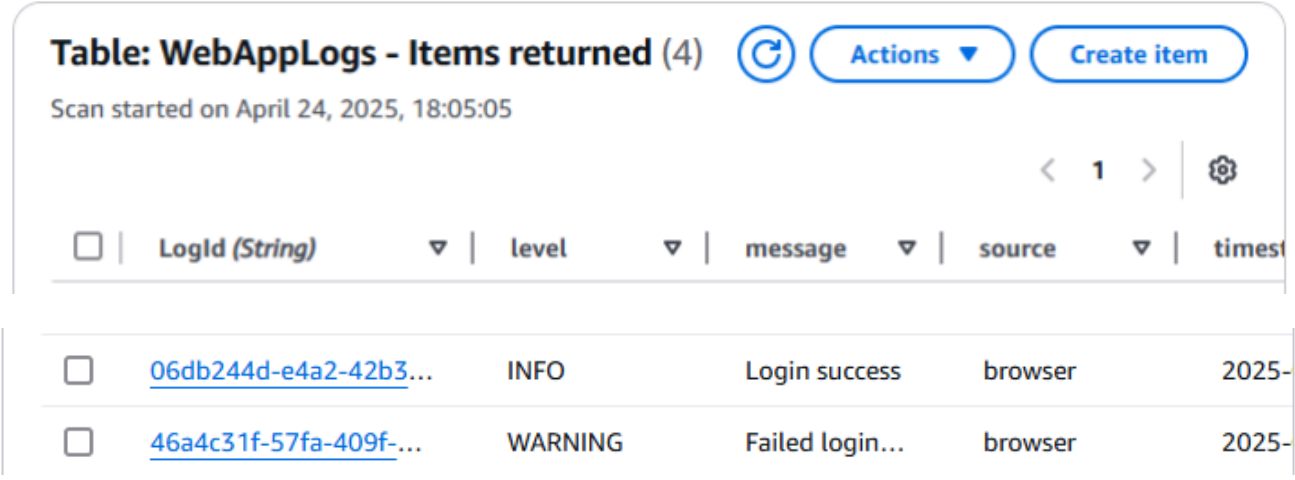


Figure 7.2.6: Test Case 5

Test Case 6:

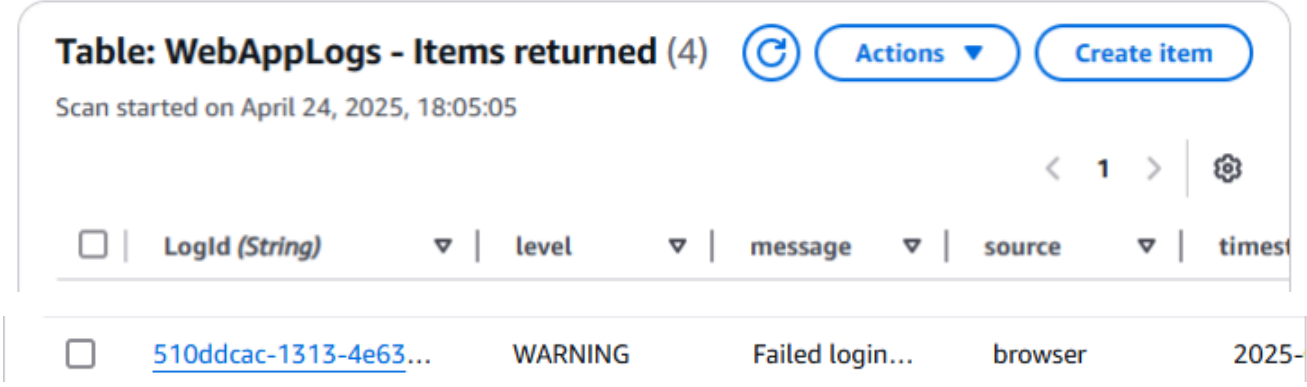


Figure 7.2.7: Test Case 6

8. OUTPUT SCREENS

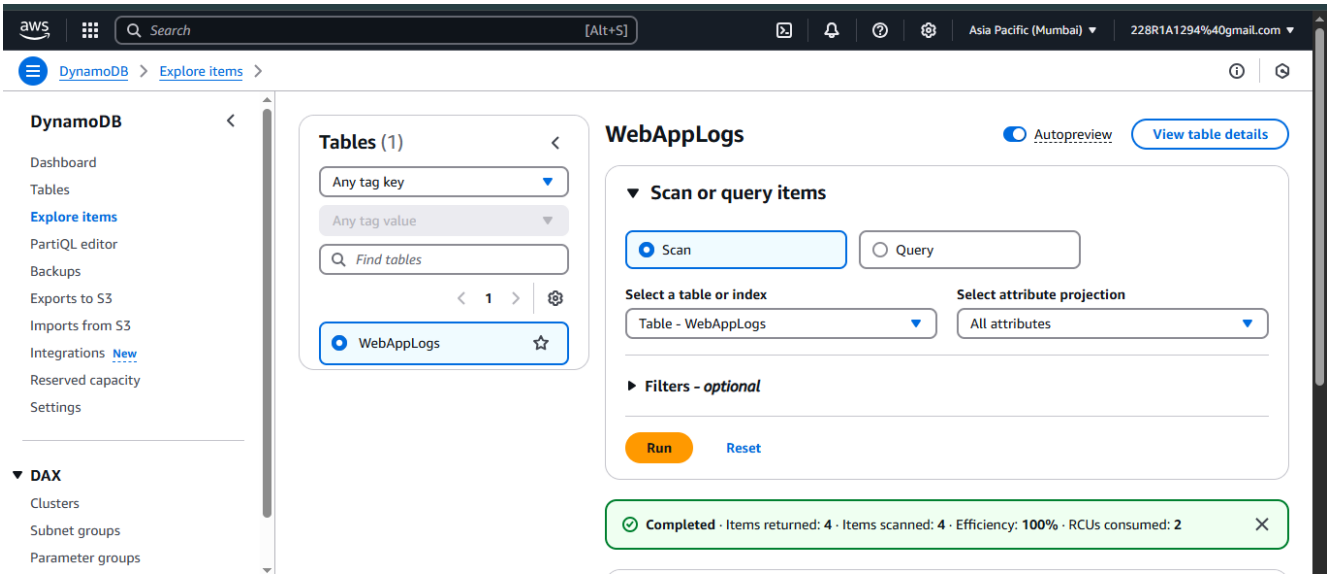


Figure 8.1: Output Screen-1

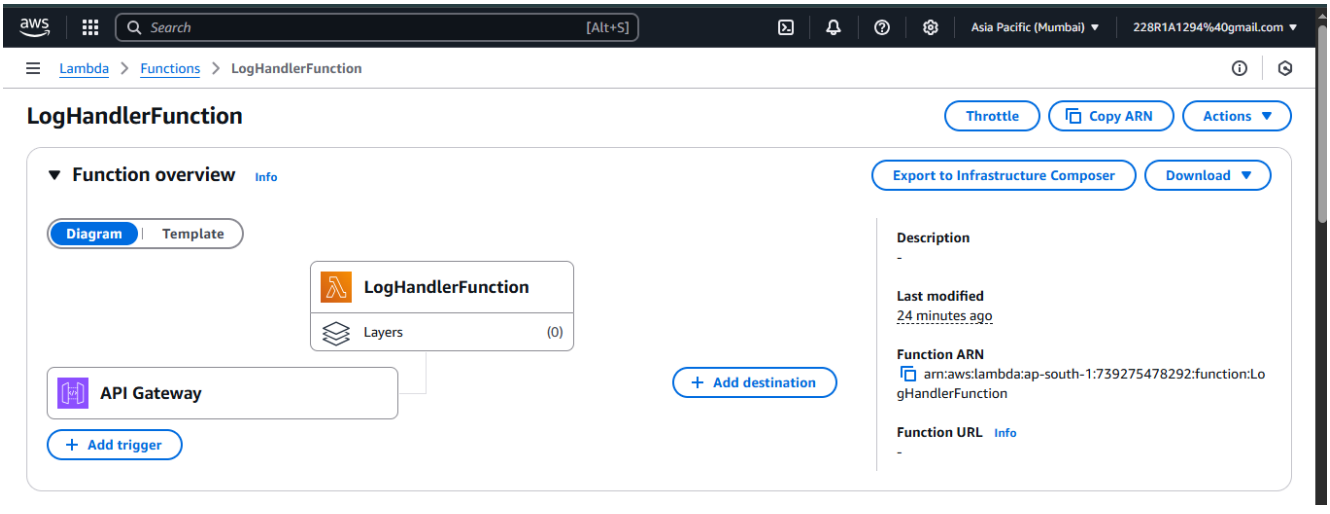


Figure 8.2: Output screen-2

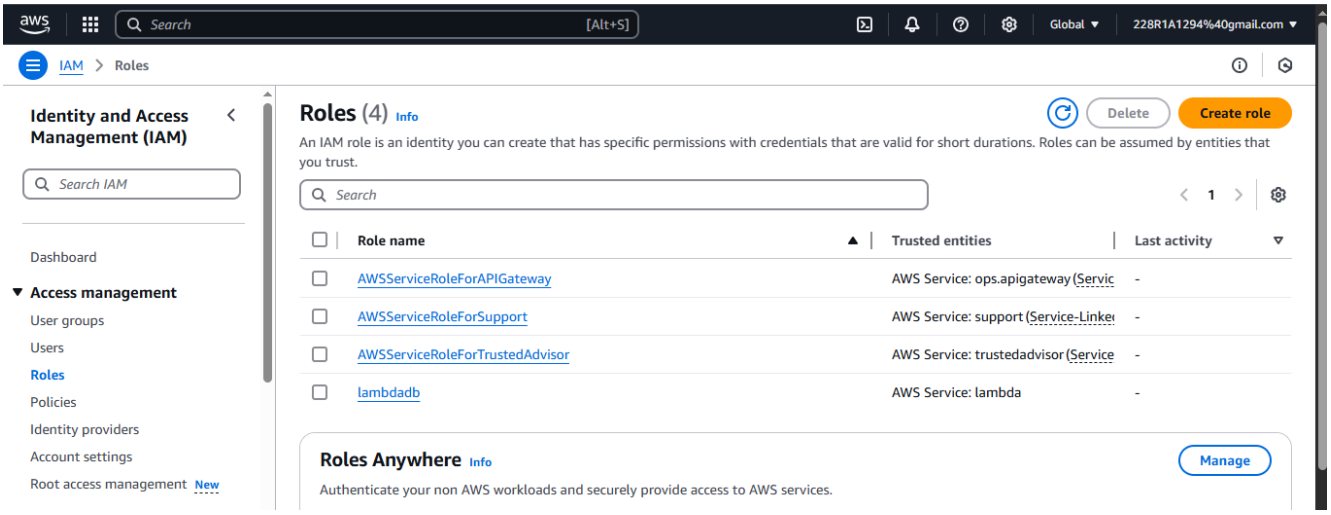


Figure 8.3: Output screen-3

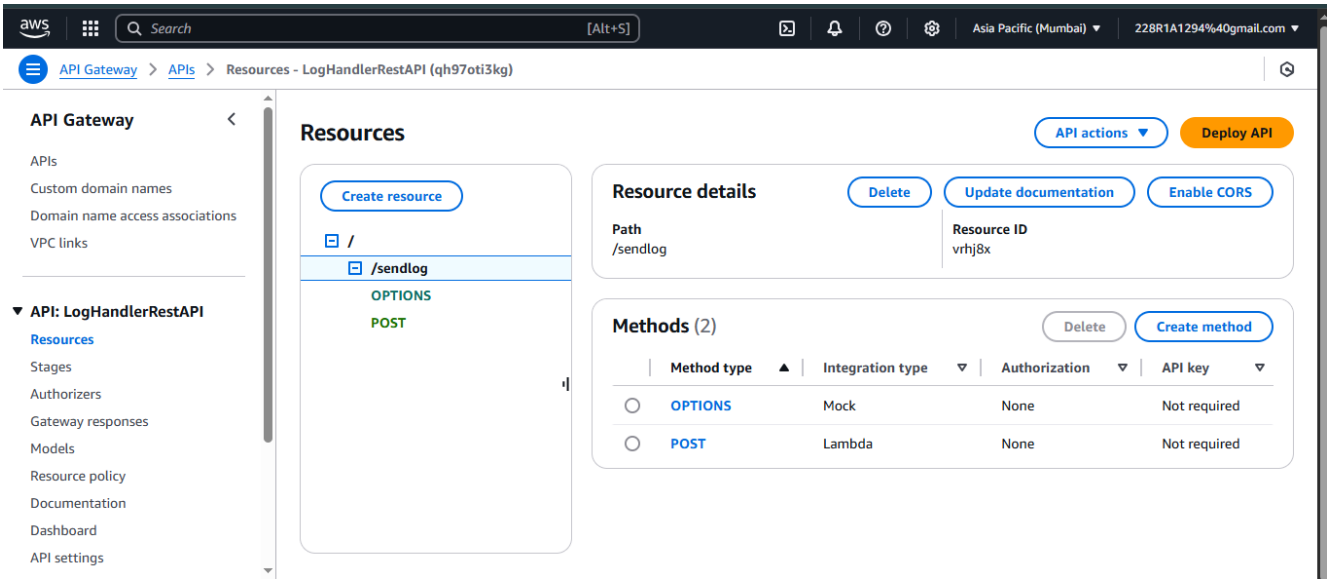


Figure 8.4: Output screen-4

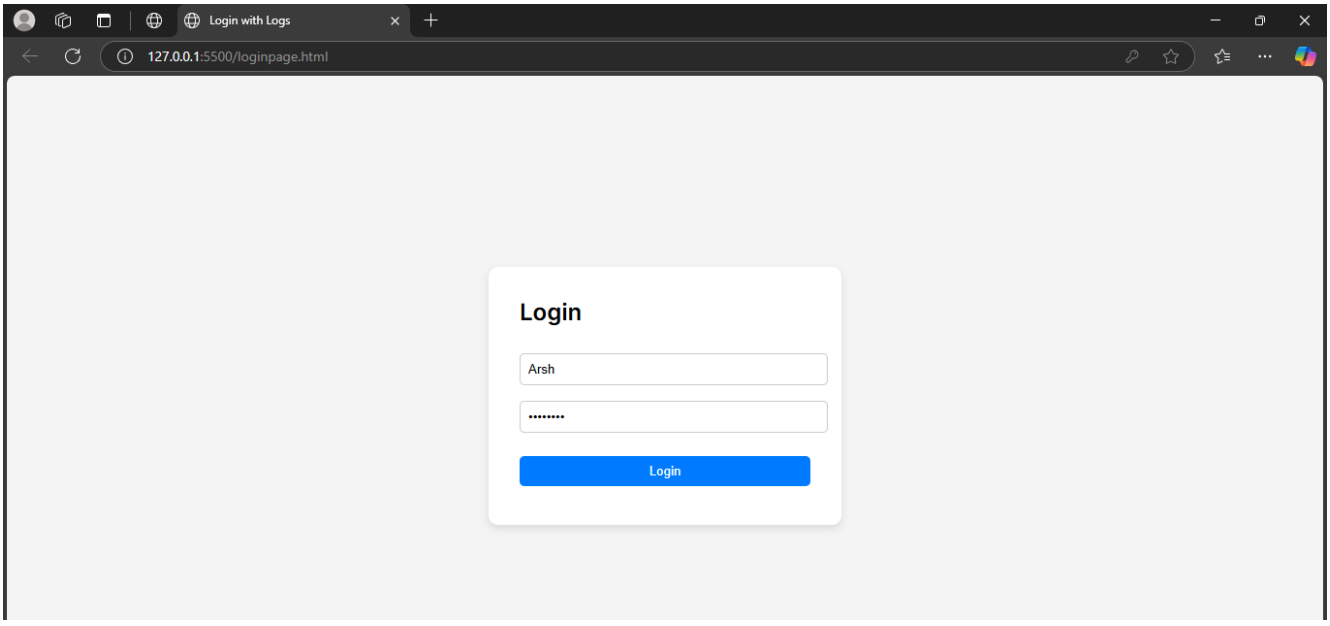


Figure 8.5: Output screen-5

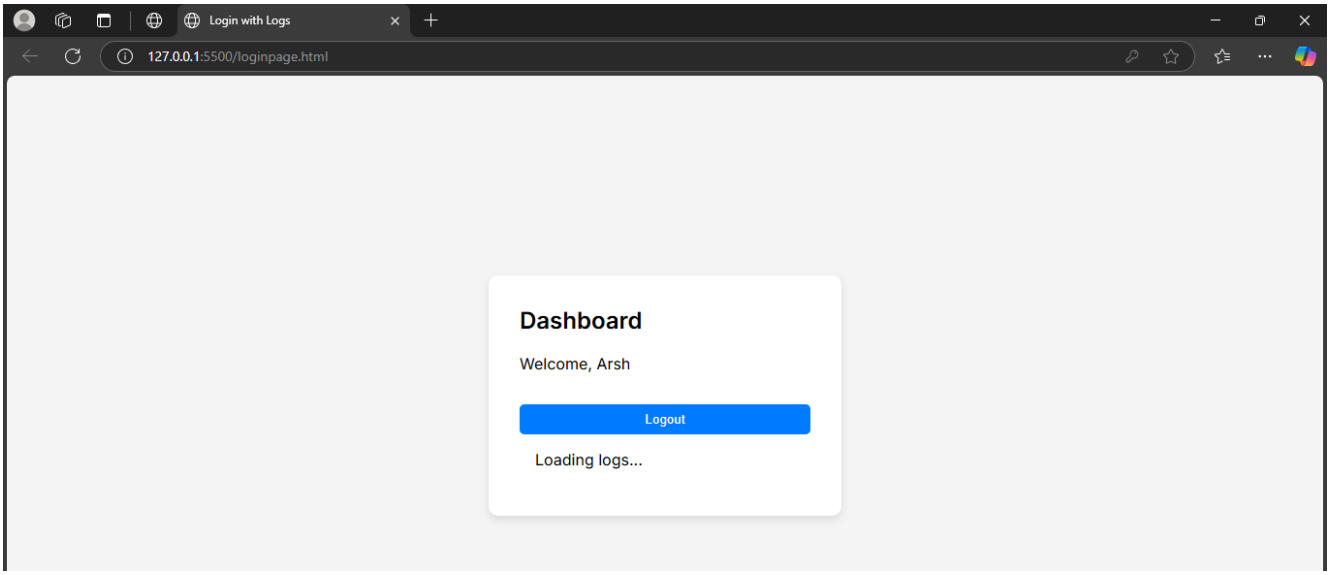


Figure 8.6: Output screen-6

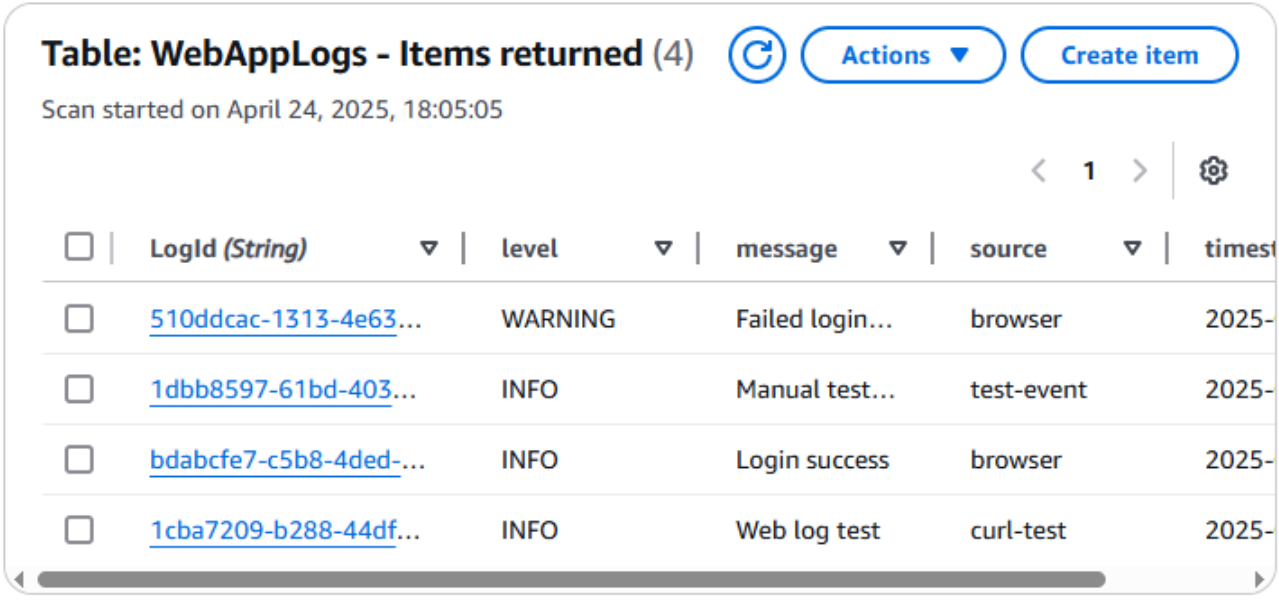


Figure 8.7: Output screen-7

	A	B	C	D	E	F	G
1	LogId	level	message	source	timestamp		
2	510ddcac-	WARNING	Failed log	browser	2025-04-24T12:34:53.340084		
3	1dbb8597-	INFO	Manual te	test-even	2025-04-24T12:23:40.787489		
4	bdabcfe7-	INFO	Login succ	browser	2025-04-24T12:34:05.694449		
5	1cba7209-	INFO	Web log t	curl-test	2025-04-24T12:33:04.796528		
6							

Figure 8.8: Output screen-8

9. CONCLUSION

The integration of AWS Lambda with DynamoDB and REST APIs presents a powerful, scalable solution for efficient and automated log processing in modern web applications. By leveraging AWS Lambda's serverless architecture, developers can process logs in real time without the need to manage infrastructure, ensuring high availability and low latency. DynamoDB, with its fast and flexible NoSQL capabilities, complements Lambda by providing a reliable storage solution that can handle massive volumes of structured log data. This combination enables seamless ingestion, storage, and retrieval of log events, making it ideal for applications requiring immediate insights or tracking user activities.

Moreover, integrating REST APIs allows for secure and controlled communication between the frontend and backend, enabling logs to be sent and processed dynamically. This architecture not only simplifies the deployment and maintenance of log management systems but also enhances observability, supports better debugging, and contributes to proactive application monitoring. As a result, the use of AWS Lambda and DynamoDB for log processing has become a strategic approach in cloud-native application development, offering improved performance, cost-effectiveness, and real-time operational intelligence. With the growing complexity of applications and the need for efficient logging, this serverless pattern is poised to become a core component in modern software infrastructure.

This architecture also aligns well with modern DevOps practices, supporting continuous deployment and monitoring pipelines by allowing logs to be integrated with tools like Amazon CloudWatch, ElasticSearch, or third-party platforms such as Datadog and Splunk. The event-driven nature of AWS Lambda ensures that log processing is both scalable and cost-efficient, as execution is only billed per invocation and duration, eliminating the overhead of always-on servers. Furthermore, DynamoDB’s seamless scalability and managed infrastructure reduce operational complexity, allowing development teams to focus on enhancing application features rather than worrying about backend maintenance. Overall, the serverless log processing system not only empowers real-time diagnostics and enhanced security monitoring but also reinforces a robust foundation for building resilient, cloud-native applications.

10. FUTURE ENHANCEMENTS

The current system effectively captures and stores client-side log events triggered by user interactions on a web application using AWS Lambda and DynamoDB. However, several enhancements can be incorporated to improve scalability, reliability, security, and analytics capabilities. One of the key future improvements is integration with AWS CloudWatch or Elasticsearch to support advanced log analysis and visualization. While DynamoDB serves well for structured storage, integrating these tools can enable powerful real-time monitoring, filtering, and dashboard creation for better operational insight. Another enhancement is to implement role-based access control (RBAC) to ensure only authorized services or users can trigger or view logs. This would improve security, especially when scaling the system for multi-user environments or enterprise applications. Currently, logs are captured from a single client interface. Expanding the solution to support multiple microservices or APIs can help centralize log management across various modules in a larger application. This would involve updating the log schema and modifying Lambda logic to dynamically route logs based on context. Furthermore, error classification and alerting could be added. For example, Lambda can be integrated with Amazon SNS or SES to send email/SMS alerts when a critical error log is received. This allows administrators to respond proactively to system failures. Another significant enhancement is the addition of machine learning models to detect anomalies in log patterns. This could help in predicting potential issues or security threats before they become critical. These models can be trained using historical log data and hosted using Amazon SageMaker. Lastly, data archival and backup mechanisms can be implemented. Although DynamoDB is highly available, long-term log data could be periodically backed up to Amazon S3 and archived to Amazon Glacier to reduce costs and preserve historical logs. These future enhancements aim to transform the project from a basic log collector into a robust, scalable, and intelligent logging ecosystem ready for real-world deployment in production systems.

11. REFERENCES

[1] Sharma, A., Gupta, R., Kumar, S., & Jain, P. (2023). AWS Lambda for Real-Time File Processing and Notifications in Cloud Storage Systems. 2023 International Conference on Cloud Computing and Big Data (CCBD), Pune, India.

[2] Patel, M., Nair, S., & Verma, A. (2023). Enhancing Email Notifications and Alerts in Cloud-Based Systems Using Amazon SES and SNS. 2023 International Conference on Cloud Services and Technology (ICCST), New Delhi, India.

[3] Yadav, R., Rao, P., & Mehta, T. (2022). Security and Scalability Considerations in AWS Lambda-Based Notification Systems. 2022 International Conference on Cloud Security and Innovations (ICCSI), Bangalore, India.

[4] Das, K., Bhat, A., & Joshi, M. (2021). Automating Cloud File Upload Notifications Using AWS Lambda and CloudWatch. 2021 International Conference on Automation in Cloud Computing (IACC), Hyderabad, India.

[5] Singhal, A., Patil, N., & Joshi, S. (2021). Streamlit: A Framework for Building Interactive Web Applications in Python. IEEE International Conference on Software Engineering and Applications (ICSEA), Mumbai, India.

[6] Reddy, S. R., Rani, V. S., Yadav, P., & Kulkarni, M. (2020). Automated Text Summarization Using NLP and AWS Lambda. 2020 International Conference on Natural Language Processing (ICNLP), Chennai, India.

[7] Mehta, K. R., Kumar, S., & Yadav, V. (2020). Optimizing Cloud Function-Based Architectures for Scalable Event-Driven Systems. 2020 International Conference on Cloud Function Architectures (ICFA), Kolkata, India.

[8] Sharma, M., Patel, V., & Gupta, R. (2019). Enhancements in Cloud-Based Notification Systems Using SNS and Lambda. 2019 International Conference on Cloud Technology and Applications (ICTA), Jaipur, India.

[9] Kapoor, A., Das, M., & Kumar, T. (2019). Real-Time Event-Driven Notification Systems in AWS Cloud. 2019 IEEE International Conference on Cloud Computing and Services (ICCCS), Delhi, India.

[10] Amazon Web Services (2023). Building Event-Driven Architectures with AWS Lambda. AWS Whitepaper.
https://docs.aws.amazon.com/lambda/latest/dg/lambda-introduction.html

[11] Amazon Web Services (2022). Serverless Architectures with AWS Lambda and DynamoDB. AWS Whitepaper. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html

[12] Sato, K., & Nguyen, H. (2021). Scalable Serverless Logging Architecture. ACM Transactions on Cloud Computing.

[13] Pereira, F., & Rocha, J. (2021). Real-Time Event Logging Using Cloud Functions and NoSQL. IEEE CloudCom.

[14] Zhang, X., & Chen, Y. (2020). Performance Analysis of Serverless Workflows in AWS Lambda. International Journal of Cloud Computing.

[15] Leclercq, P., & Gautier, M. (2020). Asynchronous Logging in Serverless Systems. 2020 IEEE Cloud.

[16] Kumar, N., & Arora, D. (2019). Event-Driven Computing for Cloud Applications. 2019 Cloud Engineering Conference (CEC).

[17] Rajan, A., & Rathi, K. (2019). Integrating DynamoDB with Serverless Apps for Real-Time Analytics. ACM International Conference on Data Engineering.

[18] Jindal, V., & Menon, L. (2018). Monitoring User Behavior Logs Using CloudWatch and Lambda. 2018 IEEE CloudComp.

[19] Tripathi, R., & Deshmukh, A. (2018). Serverless Architecture for Intelligent Log Analytics. IEEE Cloud Security Journal.

[20] Sundar, S., & Lal, V. (2018). Scalable Logging and Monitoring in Cloud-Native Systems. 2018 ACM Conference on Distributed Computing.

[21] Tao, L., & Zhou, M. (2017). Function-as-a-Service Logging for Microservices. IEEE International Conference on Microservice Architectures.

[22] Jansen, B., & Keller, J. (2017). Stream-Based Logging Systems in AWS. 2017 International Workshop on Stream Processing.

[23] Batra, S., & Kaur, I. (2016). NoSQL Databases for Real-Time Serverless Event Logging. 2016 International Journal of Advanced Cloud Computing.

[24] Fernandez, J., & Silva, P. (2015). AWS Lambda-Based Error Logging for Web Applications. IEEE CloudTech.

[25] Thakur, D., & Mishra, R. (2015). Performance and Logging in Serverless Frameworks. 2015 Cloud Performance Conference.

[26] Chandra, S., & Gupta, R. (2014). Using Cloud Functions for Structured Log Management. ACM Digital Library.